

The Move Borrow Checker

Abstract. The Move language provides abstractions for programming with digital assets via a mix of value semantics and reference semantics. Ensuring memory safety in programs with references that access a shared, *mutable* global ledger is difficult, yet essential for the use-cases targeted by Move. The language meets this challenge with a novel memory model and a modular, intraprocedural static reference safety analysis that leverages key properties of the memory. The analysis ensures the absence of memory safety violations in all Move programs (including ones that link against untrusted code) by running as part of a load-time *byte-code verification* pass similar to the JVM [6] and CLR [9]. We formalize the static analysis and prove that it ensures the absence of both dangling references and memory leaks.

1 Introduction

The Move language[13] provides abstractions for writing safe *smart contracts* [17, 24] via a mix of value semantics and reference semantics. Programmers can use value semantics and the `move` operator to transfer ownership of an asset to another user or to a procedure. For example, a procedure signature like `fun buy(c: Coin): Car` intuitively says: “if you give me a coin, I will give you a car”. By contrast, references enable programmers to temporarily share a value for reading or writing. The signature `fun register(c: &Car, fee: Coin): Registration` says “if you show me that you own a car and pay a fee, I will give you a registration for the car”. Passing a parameter by value transfers ownership of the value to the callee, whereas passing by reference allows the callee to temporarily borrow the value.

Designing a language that allows mixing reference and value semantics without exposing programmers to currency conservation bugs, memory safety violations, or memory leaks is challenging. For example, code like `let coin_ref = &c; pay(*c_ref); pay(move c)` violates currency conservation by “double spending” the coin `c`. More familiar memory issues like dangling references to local memory (e.g., `let x = 5; return &x`), use-after-move (e.g., `let y = move x; *x`), and `let y = &x; let z = move x;` (dangling reference to moved memory) can also arise.

The popular Rust language has pioneered a safe, yet usable approach to mixing reference and value semantics: a *borrow checker* enforces an ownership discipline that prevents memory safety issues without requiring garbage collection or manual memory management. Move builds on this foundation, but must provide two key features that are not supported by Rust: mutable global state and memory safety enforcement at the executable level.

Shared, mutable global storage An account-based blockchain is a shared, persistent, *mutable* global store. Any Move program can mutate an arbitrary number of global memory cells. For example, an account can send a payment to any other account in the system, and a payment mutates both the sender’s balance and the recipient’s balance. In addition, programs may dynamically allocate and free an arbitrary number of global cells (e.g., creating a new account will allocate a new global memory cell, and deleting an account will free a cell). Move must expose a programming model that supports both dynamic creation/destruction of global memory cells and mutable references to global state, but without sacrificing memory safety.

By contrast, a Rust program has a fixed number of immutable global (`static`) variables. These variables must be initialized to constant expressions and cannot be reassigned during program execution. This does not provide sufficient flexibility for implementing shared, mutable global storage.

Memory safety at the executable level Smart contracts deployed on a blockchain store digital assets with real-world value, but must tolerate arbitrary interactions with untrusted code running in the same address space. Even if a deployed contract is memory-safe, an attacker can intentionally write memory-unsafe code that attempts to break the integrity of the deployed contract. Thus, Move must prevent all memory safety issues in *both* the deployed code and the attacker code. This differs from Rust, where the borrow checker can intentionally be subverted via mechanisms like `unsafe`.

Contributions Move tackles these problems by simplifying the Rust memory model to enable a canonical representation of reference values, then adding extensions to support a mutable global store. The key simplification is that Move memory is structured as a forest of tree-shaped values. The roots of this forest are local variables on the call stack. Move allows the creation of references to values and embedded sub-values, but (in contrast to Rust) does not allow references to be embedded inside a value. This eliminates the need to associate reference values with lifetimes, which is a significant source of complexity in the Rust borrow checker. Instead, a Move reference value can be canonically represented as an *access path* [4] rooted in a local variable.

The Rust community has struggled to find a clean formalization of the borrow checker (see Section 5). However, Move’s canonical references suggest a natural abstract domain for its borrow checker: a set domain of access paths associated with each storage location. With this information in hand, borrow checking analysis is reduced to a set-based check at each instruction that writes a storage location or reference. We formalize the static analysis and prove that it ensures the absence of both dangling references and memory leaks on Move bytecode programs.

To support mutable global state, Move adds a new kind of root in the tree-shaped memory: a global memory cell indexed by a (statically known type, dynamically determined account address) pair. This provides expressivity required

for smart contract programming without adding significant complexity to the borrow checker.

Both the Move bytecode verifier and the Move compiler implement the borrow checker analysis. The bytecode verifier which is run whenever new bytecode is loaded into the Move virtual machine, in a manner similar to the JVM [6] and CLR [9]. This enforces memory safety for all loaded bytecode regardless of provenance. The compiler borrow checker ensures that generated code will always pass the verifier, and surfaces reference-related errors to programmers in an easy-to-understand fashion. These implementations were originally developed for Libra/Diem [1], and are now used by developers in a number of public blockchain platforms including Sui [22], Aptos [19], StarCoin [21], and OL [18].

2 Language Design for Reference Safety

In this section, we will first give an overview of memory management in the Move language with emphasis on the key feature for facilitating static analysis: tree-shaped values that enable us to represent reference values as structured paths (§2.1).

Next, we present a static analysis that builds on these features to prevent dangling references. We organize our discussion around the three primary challenges our analysis faces: preventing dangling references to memory in the same procedure (§2.2), to memory in different procedures (§2.3), and to global memory (§2.4). The global reference analysis leverages Move’s type-indexed global memory and encapsulation features to enforce a global property with intraprocedural checks.

Each of the analysis sections contain examples of bad Move code that must be rejected. The analysis operates on Move bytecode, but we will write our examples in Move source code for readability. Although the source language abstracts away some bytecode features such as the operand stack, we have chosen examples that capture the essence of reference issues in the bytecode.

2.1 Memory management

Move has three kinds of storage:

1. **Procedure-local variables** Each procedure frame in the call stack has a fixed set of local variables that are uninitialized at the beginning of a procedure. Initialized variables can store values and references.
2. **Shared operand stack** All procedures in the call stack share a single operand stack that can store both values and references. Procedures can use the operand stack both for local computation and to share arguments/return values with other procedures. At the beginning of program execution, the call stack holds one frame and the operand stack is empty. The same conditions must hold for a program to terminate without an error.

3. **Global storage** Move has no global variables, no heap, and cannot access the filesystem or network. However, programs can access persistent data by reading from and writing to shared global storage that persists across program executions. Global storage is organized as a partial map from 16 byte *account addresses* to record values: $(\text{Addr} \times \text{Type}) \multimap \text{Record}$. § 2.4 explains the design of and motivation for the global storage in more detail.

Values are Tree-Shaped Move has primitive data values, nominally typed records, resizable vectors, and reference values. Move programmers can create references that point into local variables or into global storage, but not into the operand stack. Reference types are tagged with *permissions*: either mutable (written `&mut T`) or immutable (written `&T`), as in Rust. References to other references (e.g., `&&u64`) are not allowed.

Both records and vectors can store primitive values, and other records/vectors, but not references. Global storage can hold records, but not references. This ensures that non-reference values and the global storage are always tree-shaped.

References are Structured Paths In a byte-addressable memory, a reference value is a dynamically determined index into the array of memory. This unstructured representation makes it difficult to reason about the relationship between two different reference values—e.g., “can writing through reference *i* change the memory pointed to by reference *j*?”

By contrast, Move storage, values, and reference-related instructions are designed to ensure that a reference can be represented as a structured access path. For example: say we have a record value $\{ f: \{ g: 1 \}, h: [2,3] \}$ of type *T*, where the $\{$ syntax denotes a record and the $[$ syntax denotes a vector. If this value is stored in a local variable *x*, we can represent a reference to the value stored by field *g* as the path $x/f/g$. Similarly, we can represent a reference stored at index 1 of the vector in field *h* as $x/h/1$. If the same record value is in global storage at account address *A*, we can represent these paths as $A/T/f/g$ and $A/T/h/1$.

A path is a canonical representation for a particular location in memory. Syntactically distinct paths refer to distinct memory locations. In addition, the structured nature of paths introduces a partial order on reference values. Two reference values either have a prefix relationship (e.g., x/f is a prefix of $x/f/g$), a suffix relationship (e.g., $x/f/g$ is a suffix of x/f), or are incomparable (e.g., $y/f/g$ and $x/f/g$). Writing to a path cannot change the memory named by incomparable paths. As we will see, our reference safety analysis takes advantage of these nice properties to simplify static reasoning about code that uses references.

2.2 Preventing Dangling References to Locals

We begin our discussion of the reference safety analysis with a pair of code snippets that exemplify the problems the analysis must prevent. The code comments in the snippet show the abstract state of the analysis, but we will ignore them at first.

```

fun dangle_after_move(c: Coin) {
  let r = &c.f; // r ↦ {c.f}
  let x = move c; // †
  let y = *r; // read from dangling ref!
}

let v = vector[5];
let v_ref = &mut v; // v_ref ↦ {v}
let ref = &mut v[0]; // ref ↦ {v*}, v_ref ↦ {v}
*v_ref = vector[]; // †
*ref = 10; // write to dangling ref!

```

Fig. 1: Two examples of creating dangling references.

The first program in Figure 1 creates a reference `r` to the `f` field of formal parameter `c`, *moves* the value stored in `c` into `x` and then reads `r`. The `move` assignment works by assigning the value bound to `c` to `x` and then “clearing” `c` by assigning it to $\perp$.

The ability (and in some cases, requirement) to move values instead of copying them is a key feature of the Move language—it prevents “double spending” of monetary values like `c`. However, the `move` causes the reference value stored in `r` to become dangling.

Similarly, if the programmer creates a reference to a value that is overwritten via a destructive update, a dangling reference may result. The second program in Figure 1 creates a vector `v` of size 1, acquires a reference `ref` into index 0 of the vector, and then reassigns `v` to an empty vector via the write to `v_ref`. As a result, the write `*ref` accesses a dangling reference.

Ownership and Borrowing Our analysis enforces a programming discipline based on ownership to prevent the problems above. A location in memory (either a local variable, stack location, or global key) of type `T` is the *owner* of the value it stores. A value of type `T` can only be moved or (if `T` is a mutable reference) written via its owning location. However, a value stored in a local or global can be *borrowed* from its owner by creating a reference to it (e.g. `&v`) or extending an existing reference (e.g. `&ref.f`). The analysis will not allow the value to be moved or written until all borrows have ended (i.e., the reference values produced by the borrows have been destroyed). This discipline has a natural recursive structure: if `y` borrows from `x` and then `z` subsequently borrows from `y`, `x` does not regain ownership until both the `y` and the `z` borrows have ended.

As we will show in §4, a program that follows these rules cannot create a dangling reference by writing path `p` when a reference path that is a strict suffix of `p` exists elsewhere or moving a variable `x` when a reference path that is a suffix of `x` exists elsewhere. As a side benefit, this discipline ensures the absence of memory leaks.

Borrow Analysis Mechanics As we explained in §2.1, the concrete representation of a reference value is an access path rooted in a local variable. This enables a very direct abstraction of concrete values—an abstract reference is either an exact concrete path (e.g., `x/f`), or a concrete path with an exact prefix and an unknown, possibly-empty suffix of arbitrary length (e.g., `x/f/*`). The analysis tracks the set of abstract references that may be stored in each local variable or stack location (non-reference values are encoded as the empty set).

```

fun ret_local(): &u8 { let x = 7; &x /* Si ↦ {x} */ } †
fun ret_param(x: u8): &u8 { &x /* Si ↦ {x} */ } †
fun ret_ref_param(x: &u8): &u8 { x /* x ↦ {x}, Si ↦ {x} */ }
fun ret_param_fld(s: &S): &u8 { &s.f /* s ↦ {s}, Si ↦ {s.f} */ }

```

Fig. 2: Examples of reference behavior in procedure calls.

At each mutating instruction (either a move of a local, an assignment to a local, or a write to a reference), the analysis compares the abstract paths being mutated to abstract paths stored elsewhere in locals or on the stack. The analysis only allows the instruction to succeed if each mutated path is not a prefix of any outstanding path.

Returning to our examples in Figure 1, each instruction is annotated with the abstract post-state (a mapping of local variables to abstract references) inferred by the borrow checker analysis. A † indicates that the borrow checker would reject the program after processing the instruction. The first program is straightforward: the `&c.f` instruction creates a borrow of `c`, then the analysis rejects the program at `move c` because the mutated path `c` is a prefix of the outstanding path `c.f`.

In the second program, the local `v_ref` stores a reference to the path `v`. The next instruction `let ref = &mut v[0]` associates the path `v*` with `ref`. This is our first encounter with abstraction in the analysis: the `*` reflects that the analysis does not know which offsets of `v` have been borrowed by `ref`. Finally, the analysis chooses to reject the write to `v_ref` because the mutated path `v` is a prefix of the outstanding path `v*`.

2.3 Borrow Discipline Across Procedures

Move procedures can accept reference arguments and return references, which introduces new ways to create dangling references. However, we can extend the borrow discipline and its corresponding analysis to prevent these as well.

First, we consider the problem of returning dangling references. Each procedure on the left of Figure 2 returns a reference value to its caller by pushing the return value on the operand stack (we write S_i for the i th operand stack slot) and then executing the `Ret` instruction (not shown). The first two procedures return dangling references to local variables of the procedure, but the second two return safe references to memory that will outlive the procedure.

Each procedure shows the abstract state before the procedure returns. The analysis will not allow a return value to have any borrow of a local variable (`ret_local`) or a formal with a non-reference type (`ret_param`). The `ret_ref_param` procedure is safe because it has no borrows—it returns the reference value bound to `x` by the caller. Finally, `ret_borrowed_param` has a borrow of formal `s`, but this is ok because `s` is a reference parameter rather than a local.

Procedure Calls Require Ownership Lastly, we consider the problem of handling procedure calls. Our static analysis is modular, so it must soundly summarize the effects of a procedure call with no information other than the callee’s type signature. A reference value returned by a callee is conservatively assumed to be borrowed from all of the procedure’s reference arguments, with extra care to avoid conflating immutable and mutable references (see the `Call` rule in §4 for details).

A subtle consequence of the borrow discipline described in the previous section is that every non-reference value pushed on the operand stack has no outstanding borrows. Since procedures pass arguments to their callees on the stack, this ensures that a caller cannot retain references to a value passed to a callee. With this possibility out of the way, the only danger that remains is a dangling reference caused by a reference value written by a callee. To prevent this, the analysis enforces a single rule: an argument with a mutable reference type must not have any outstanding borrows.

The two examples on the right of Figure 2 show unsafe call sites. In both cases, the arguments are evaluated left to right and pushed on the operand stack before the `Call` instruction. The abstract states shown are at the program point after the arguments have been pushed, but before the call instruction. The analysis rejects each example because stack slot S_1 holds an abstract path that is a prefix of a different abstract path stored in stack slot S_0 .

In addition to preventing dangling references, this strategy for handling calls ensures a very useful property: **a mutable reference parameter cannot alias with any other parameter!** Eliminating aliased mutable data allows Move programmers to write procedures without defensive checks for aliasing and greatly facilitates precise and scalable static program verification [14].

2.4 Dangling References to Global Memory

We conclude our informal presentation of the analysis by considering the thorny problem of allowing mutable access to global memory while preventing dangling references. At first blush, this might seem impossible to do with a modular analysis—the whole point of global memory is that you can access it from anywhere! However, Move’s global memory instructions are carefully designed to enable local reasoning about the safety of global memory accesses. The two analysis extensions described in this section build on this design to prevent dangling references to global memory using the existing borrow analysis machinery.

Global Memory Indexed By Encapsulated Types Move’s global memory is a partial map indexed by a pair of a statically chosen record type T and a dynamically chosen account address a . A value stored at key (T, a) is a record of type T . Record types are declared in *modules* consisting of type and procedure declarations. The following table summarizes the Move global state operations:

<code>move_to<T>(a, T)</code>	Publish T at address a
<code>move_from<T>(a): T</code>	Remove T from a
<code>&<T>(a): &mut T</code>	Get mutable ref to T at a

```

fun addr_alias(a1: address, a2: address)
  acquires T {
    let t_ref = &<T>(a1); // t_ref ↦ {T}
    let t = move_from<T>(a2); // †
    *t_ref = ... // access a dangling ref
  }

fun borrow_remove(a: address) acquires T {
  let t_ref = &<T>(a); // T  $\xRightarrow{\epsilon}$  t_ref
  let t = remove_t(a); †
  *t_ref = ... // access a dangling ref
}

fun remove_t(a: address): T acquires T {
  move_from<T>(a)
}

```

Fig. 3: Examples of creating dangling references via global storage operations

Programmers can publish a value of type T to an address in global state, remove the value of type T stored at an address, and acquire references to a value already published in global state. Modules encapsulate access to their declared types; in particular, the global state access operations can only be used on a type declared *inside the current module*.

The decision to include a type in these operations simplifies static reasoning about aliasing of locations in global memory. Two global access operations involving keys $(T1, a1)$ and $(T2, a2)$ can only touch the same memory if $T1 = T2$ and $a1 = a2$. Combined with type encapsulation, this means that global accesses in distinct modules touch distinct memory by construction. Without this property, a local reference safety analysis would not be practical.

Abstracting Dynamic Global Accesses Move instructions index into global memory using an `address` value chosen at runtime. This means that code like `addr_alias` in Figure 3 may create a dangling reference if `a1` and `a2` are bound to the same address value. Similarly, performing a `move_to<T>(a, T)` while a global reference created by a `&<T>(a)` is still active creates a dangling reference.

Rather than attempting to reason about address equality, our analysis conservatively assumes that all global accesses indexed by type T touch the same address. This decision suggests a simple extension to the borrow checker: a dummy T parameter that abstracts all concrete cells in global memory keyed by type T . In the `addr_alias` procedure of Figure 3, the analysis introduces a path rooted in T at the `&<T>(a)` instruction and rejects the program at `move_from<T>` because there is an outstanding reference to T . This is exactly how the analysis deals with programs like `let x = &y; let z = move y` where `y` is a local, only with T in place of `y` and global borrow/move instructions in the place of local ones.

Global Accesses Across Procedure Boundaries A similar, but more insidious problem arises if the global accesses occur in procedures with a caller/callee relationship, as `borrow_remove` procedure in Figure 3 illustrates. Fortunately, the `acquires` annotation on `remove_t` allows the borrow analysis to reject this program—a call of a procedure marked with `acquires T` is treated as a move of T .

A separate intraprocedural analysis checks that any procedure containing a `&<T>` or `move_from<T>` instruction is annotated with `acquires T`. Any procedure


```

module M1 {
  fun f(a: address): &mut T acquires T {
    &<T>(a)
  }
  fun g(a: address): T acquires T {
    move_from<T>(a)
  }
}

module M2 {
  fun bad(a: address) {
    let ref = M1::f(a);
    let t = M1::g(a); // ref now dangling
  }
}

```

Fig. 4: Dangling reference via leaking a global reference to T outside its declaring module.

that invokes an `acquires` T-annotated procedure in the same module must also be annotated. A procedure that calls an `acquires` procedure declared in a different module need not be annotated.

Global References Cannot Be Returned The scheme described so far prevents dangling references to a global memory cell of type T inside the module that declares T. One issue remains: leaking a reference to a global cell of type T outside of T’s declaring module, as illustrated in Figure 4.

The possibility of this leakage undermines our efforts to modularize reasoning about reference invalidation. Thus, our analysis prevents it by banning returns of references to global memory. This happens implicitly by following the return discipline described in §2.3: only borrows of reference parameters may remain on the stack when a procedure returns, and a global is not a reference parameter.

3 Move Operational Semantics

In this section, we define a small-step structured operational semantics for a subset of the Move language chosen to illustrate the key challenges of reference safety. We begin with some preliminary notation and definitions.

Partial functions and lists We use partial functions to represent record values and for mappings that are parts of semantic states. Lists are used for sequences of field accesses and for the stack component of semantic states.

Following common convention, if $f : A \rightarrow B$ is a partial function, then $\text{dom}(f)$ is its domain and $\text{img}(f)$ is its range. We write $f[a \mapsto b]$ for the function that is the same as f on every argument except a , and which maps a to b . Similarly, $f \setminus a$ is the partial function equivalent to f except that it is undefined at a .

We write $[]$ for the empty list and $e :: l$ for the result of placing e at the front of list l . Similarly, $l :: e$ is the list with e appended to l and, by slight overloading of notation, $l :: l'$ is the concatenation of lists l and l' . If ℓ is a list, we write $\ell[i]$ for the i th element of list ℓ . We use $\text{len}(\ell)$ to denote the length of a list.

Types Let $PrimType$ be the set of primitive types, including the types `Bool` for Boolean values, `Int` for integers, and `Addr` for account addresses. Let F be a fixed, finite set of *field names*. The set $ValType$ of value types is the least set satisfying: (1) $PrimType \subseteq ValType$; and (2) if $f_1 \dots f_n \in F$ are pair-wise distinct and $t_1 \dots t_n \in ValType$, then $\{(f_i, t_i) | 1 \leq i \leq n\} \in ValType$. Let $ImmRefType = \{Imm(t) \mid t \in ValType\}$, $MutRefType = \{Mut(t) \mid t \in ValType\}$, $RefType = ImmRefType \cup MutRefType$, and $Type = ValType \cup RefType$.

Values We define the set of values used in computation inductively. Let $PrimVal$ be the set of primitive data values. The set Val of values is the least set satisfying: (1) $PrimVal \subseteq Val$; (2) if $f_1 \dots f_n \in F$ are pair-wise distinct and $v_1 \dots v_n \in Val$, then $\{(f_i, v_i) | 1 \leq i \leq n\} \in Val$. The judgment $v : t$, indicating that value v has type t , is defined in the natural way, based on associating each primitive data value with a primitive type.

Paths A *relative path* is a possibly empty list of field names, possibly ending in the special symbol, $*$. A relative path represents a way to start at any location in memory and follow a sequence of field selections to reach a value. The symbol $*$ indicates an unknown (possibly empty) sequence of additional field names. We will only use $*$ in program analysis, not in the execution rules of the operational semantics. The borrow checker will also use rooted paths, which consist of a starting point (such as a local variable) and a relative path to be followed from that root.

It is helpful to visualize a value as a labeled tree whose edges are labeled by field names. In this visualization, a primitive value is a single leaf. The tree associated with a record value consists of a node with a subtree for each record component. If r is a record value, then for each $(f, v) \in r$, the edge from r to the subtree for v is labeled by f .

Two useful operations on values are (1) the subtree $v[p]$ of v located at relative path p , and (2) the tree $v[p := v']$ obtained by replacing the subtree reached by path p with tree v' . Using the association between values and trees, $v[p]$ is the subtree reached by path p from root v , and $v[p := v']$ is the tree with the subtree $v[p]$ replaced with v' .

Programs A *program* $\mathcal{P}$ maps each procedure name ρ to a tuple $\mathcal{P}[\rho] = \langle C, I, O, L \rangle$ consisting of a nonempty sequence of bytecodes $\mathcal{P}[\rho].C$, sequences of input types $\mathcal{P}[\rho].I$ and output types $\mathcal{P}[\rho].O$, and local variable types $\mathcal{P}[\rho].L$. We will sometimes use the shorthand $\rho.ins$ for input types (instead of $\mathcal{P}[\rho].I$), and similarly for output types $\rho.outs$ and local variable types $\rho.locals$. Each of these sequences has a known, fixed size and is indexed via square bracket notation (e.g., $\rho.ins[i]$ is the type of the i th input to ρ).

Concrete states A *concrete state* s is a pair $\langle P, M \rangle$, where P is a call stack and M is a memory. Over an unstructured set Loc of locations; a memory is a partial map from Loc to Val . Each frame is a tuple comprising a procedure name ρ , a program counter ℓ , a local store L mapping variables to locations or references and a local operand stack S that is a list of values and references.

Locations and references A *reference* r is a pair $\mathbf{ref}\langle c, p \rangle$, where $c \in \text{Loc}$ and p is a relative path that does not end in $*$. Intuitively, a reference is an idealized form of pointer into memory. When two references $\mathbf{ref}\langle c, p_1 \rangle$ and $\mathbf{ref}\langle c, p_2 \rangle$ occur in the same program, an assignment through one (*e.g.*, **WriteRef** with $\mathbf{ref}\langle c, p_1 \rangle$ on the operand stack), may affect the other only if one path is a subsequence of the other. Two references starting from distinct locations are independent; no assignment using one will change the value in memory for the other.

Because local variables of type location must be initialized, one of the instructions, **StoreLoc** $\langle x \rangle$, may set the value of a variable to a new location. However, the number of variables used in each procedure is determined statically; this number is used in the actual language implementation. In our idealized operational semantics, we use a shorthand. Specifically, we assume a fixed map LocMap that selects a distinct location based on two arguments, the height (number of call frames) of the runtime stack and a variable name. We write c_x in the operational semantics for $\text{LocMap}(\ell, x)$, where ℓ is the depth of the program call-stack at the point where this instruction is executed. Because of the dependence on stack height, recursive calls will use different locations for local x in each call. However, if x is moved to the operand stack by **MoveLoc** $\langle x \rangle$, and then later re-initialized in the same procedure invocation, the same c_x will be re-used.

Semantics Figure 6 defines the operational semantics as a small-step relation $\mathcal{P} \vdash \langle P, M \rangle \rightarrow \langle P', M' \rangle$. The first four rules define execution of procedure call, return and conditional branch instructions that manipulate the call stack or program counter. **Call** establishes a new frame at the top of the call stack which **Ret** subsequently removes. Conditional **Branch** $\langle \ell_1, \ell_2 \rangle$ pops a Boolean value from the top of the operand stack and updates the program counter to ℓ_1 if the value is true and to ℓ_2 otherwise.

For the remaining local instructions, the **Local** semantic rule defines program execution by reference to simpler *local operational semantics* where each instruction may modify the top (local) stack frame and global memory, as described below.

The **MoveLoc**, **CopyLoc**, and **StoreLoc** instructions move or copy values between local variables and the operand stack; **BorrowLoc**, **BorrowField**, **ReadRef**, and **WriteRef** to operate on reference values stored on the operand stack. The **FreezeRef** rule converts a mutable reference into an immutable one. This instruction is a no-op in the concrete semantics, but performs several important checks in the analysis (see Section 4). **Pack** and **Unpack** to create and destroy record values by binding values on the stack to fields or pushing values bound to fields onto the stack (respectively). **Pop** destroys a value on the operand stack. The generic **Op** instruction represents all arithmetic and bitwise operations that use the operand stack.

Correctness Several static requirements are needed for correct program execution. Some are easily handled by straightforward syntactic checks. For example, every bytecode instructions must be in the valid bytecode range, branch targets must be legal indices, and the last instruction must be either **Ret**. Further, each

instruction must operate on the correct number of values of appropriate type, producing the appropriate result, and updating the state without violating reference safety. The last requirement is the subject of the next section.

4 Move Borrow Checker

The borrow checker is based on modular reachability analysis, determining statically a supset of the possible overlaps between different references to memory. This reachability analysis supports run-time memory management that eliminates memory leaks and dangling pointers without garbage collection. The reachability analysis is based on a set of rules for propagating static assertions through individual instructions. These rules are applied successively to each instruction in a procedure, beginning with an abstract state characterizing the local variables and parameters passed into the procedure when it is first called. Because of loops and merge points for conditionals, an iterative algorithm replaces static assertions by weaker ones until the analysis converges. However, in our soundness proof, we assume that code is annotated with the result of this iterative algorithm; we do not model the algorithm itself, only its final result. We prove that abstract states consistent with the static rules are sound and sufficient to prevent memory leaks and dangling pointers.

More formally, we assume that each procedure $\mathcal{P}[\rho] = \langle C, I, O \rangle$ is statically analyzed to produce an abstract local state $\mathcal{P}[\rho].T[i]$ for each offset $i \in 0..len(\mathcal{P}[\rho].C)$. This abstract state represents conditions that any concrete state will be shown to satisfy at the program point before the instruction $\mathcal{P}[\rho].C[i]$ is executed.

4.1 Abstract domains

An *abstract path* consists of an abstract reference and a relative path that may end with the special symbol $*$. An abstract reference represents either a reference passed as a procedure parameter or the location of a local variable created within a procedure that holds a reference. The relative path indicates further field selections from that reference. More formally,

$$\hat{p} \in \text{AbsPath} = \text{AbsRef} \times \text{RelativePath}$$

where AbsRef includes $rp_1, \dots, rp_k$ for the k formal parameters of the procedure and rx_i for each local variable x_i that can be initialized using **StoreLoc** $\langle x \rangle$ to a distinct location associated with that variable name for this procedure invocation.

An *abstract value* is a set of abstract paths together with a reference qualifier:

$$\begin{aligned} \hat{P} \in \text{AbsPaths} &= 2^{\text{AbsPath}} \\ r \in \text{RefQualifier} &= \{\text{mut}, \text{immut}\} \\ \hat{A} \in \text{AbsValue} &= \text{AbsPaths} \times \text{RefQualifier} \end{aligned}$$

$$\begin{array}{c}
\frac{disj(\{(r_{\mathbf{x}}, \epsilon)\}, paths(\hat{S}, \hat{L} \setminus x)) \quad \rho.local[s] = ValType}{\rho, \mathbf{MoveLoc} \langle x \rangle \vdash \langle \hat{L}, \hat{S} \rangle \rightarrow \langle \hat{L} \setminus x, \hat{L}(x)::\hat{S} \rangle} \mathbf{Val} \\
\frac{\rho.local[s] \neq ValType}{\rho, \mathbf{MoveLoc} \langle x \rangle \vdash \langle \hat{L}, \hat{S} \rangle \rightarrow \langle \hat{L} \setminus x, \hat{L}(x)::\hat{S} \rangle} \mathbf{Ref} \\
\\
\frac{}{\rho, \mathbf{CopyLoc} \langle x \rangle \vdash \langle \hat{L}, \hat{S} \rangle \rightarrow \langle \hat{L}, \hat{L}(x)::\hat{S} \rangle} \quad \frac{disj(\{(r_{\mathbf{x}}, \epsilon)\}, paths(\hat{S}, \hat{L} \setminus x)) \quad \rho.local[s] = ValType}{\rho, \mathbf{StoreLoc} \langle x \rangle \vdash \langle \hat{L}, (\emptyset, \mathbf{immut})::\hat{S} \rangle \rightarrow \langle \hat{L} [x \mapsto (\emptyset, \mathbf{immut})], \hat{S} \rangle} \mathbf{Val} \\
\\
\frac{\hat{P} \neq \emptyset \quad \rho.local[s] \neq ValType}{\rho, \mathbf{StoreLoc} \langle x \rangle \vdash \langle \hat{L}, (\hat{P}, q)::\hat{S} \rangle \rightarrow \langle \hat{L} [x \mapsto (\hat{P}, q)], \hat{S} \rangle} \mathbf{Ref} \\
\frac{\rho.local[s] = ValType}{\rho, \mathbf{BorrowLoc} \langle x \rangle \vdash \langle \hat{L}, \hat{S} \rangle \rightarrow \langle \hat{L}, (\{(r_{\mathbf{x}}, \epsilon)\}, \mathbf{mut})::\hat{S} \rangle} \\
\\
\frac{\hat{P}' = \{(r, p \dagger f) \mid (r, p) \in \hat{P}\}}{\rho, \mathbf{BorrowField} \langle f \rangle \vdash \langle \hat{L}, (\hat{P}, q)::\hat{S} \rangle \rightarrow \langle \hat{L}, (\hat{P}', q)::\hat{S} \rangle} \\
\frac{}{\rho, \mathbf{FreezeRef} \vdash \langle \hat{L}, (\hat{P}, \mathbf{mut})::\hat{S} \rangle \rightarrow \langle \hat{L}, (\hat{P}, \mathbf{immut})::\hat{S} \rangle} \\
\\
\frac{}{\rho, \mathbf{ReadRef} \vdash \langle \hat{L}, \hat{A}::\hat{S} \rangle \rightarrow \langle \hat{L}, (\emptyset, \mathbf{immut})::\hat{S} \rangle} \quad \frac{disj(\hat{P}, paths(\hat{S}, \hat{L})) \quad \hat{A} = (\emptyset, \mathbf{immut})}{\rho, \mathbf{WriteRef} \vdash \langle \hat{L}, \hat{A}::(\hat{P}, \mathbf{mut})::\hat{S} \rangle \rightarrow \langle \hat{L}, \hat{S} \rangle} \\
\\
\frac{}{\rho, \mathbf{Pop} \vdash \langle \hat{L}, \hat{A}::\hat{S} \rangle \rightarrow \langle \hat{L}, \hat{S} \rangle} \quad \frac{\hat{A}_i = (\emptyset, \mathbf{immut})}{\rho, \mathbf{Pack} \langle t \rangle \vdash \langle \hat{L}, \hat{A}_1::\dots::\hat{A}_n::\hat{S} \rangle \rightarrow \langle \hat{L}, (\emptyset, \mathbf{immut})::\hat{S} \rangle} \\
\\
\frac{t = \{(f_i, t_i) \mid 1 \leq i \leq n\} \quad \hat{S}' = ::_n(\emptyset, \mathbf{immut})}{\rho, \mathbf{Unpack} \langle t \rangle \vdash \langle \hat{L}, (\emptyset, \mathbf{immut})::\hat{S} \rangle \rightarrow \langle \hat{L}, \hat{S}'::\hat{S} \rangle} \quad \frac{\mathbf{Op} : \hat{A}_1 \times \dots \times \hat{A}_n \rightarrow \hat{A}}{\rho, \mathbf{Op} \vdash \langle \hat{L}, \hat{A}_1::\dots::\hat{A}_n::\hat{S} \rangle \rightarrow \langle \hat{L}, (\emptyset, \mathbf{immut})::\hat{S} \rangle} \\
\\
\frac{m = len(\rho'.ins) \quad n = len(\rho'.outs) \quad \forall i \in 1 \dots m, q_i = \mathbf{immut} \vee disj(\hat{P}_i, paths(\hat{S}, \hat{L}) \cup \bigcup_{j=1 \dots m, j \neq i} \hat{P}_j)}{\forall i \in 1 \dots m, \begin{cases} q_i = \mathbf{mut} & \text{if } \rho'.ins[i] = MutRef \\ q_i = \mathbf{immut} & \text{otherwise} \end{cases} \quad \hat{A}_i = \bigcup_{j=1 \dots m} (\hat{P}_j \dagger *)} \\
\frac{}{\rho, \mathbf{Call} \langle \rho' \rangle \vdash \langle \hat{L}, (\hat{P}_1, q_1)::\dots::(\hat{P}_m, q_m)::\hat{S} \rangle \rightarrow \langle \hat{L}, \hat{A}_1::\dots::\hat{A}_n::\hat{S} \rangle} \\
\\
\frac{n = len(\rho.outs) \quad \forall i \in 1 \dots n, q_i = \mathbf{immut} \vee disj(\hat{P}_i, \bigcup_{j=1 \dots n, j \neq i} \hat{P}_j) \quad \forall i \in 1 \dots n, \forall (r_j, p) \in \hat{P}_i, j < len(\rho.ins)}{\forall i \in 1 \dots n, \begin{cases} q_i = \mathbf{mut} & \text{if } \rho'.outs[i] = MutRef \\ q_i = \mathbf{immut} & \text{otherwise} \end{cases}} \\
\frac{}{\rho, \mathbf{Ret} \vdash \langle \hat{L}, (\hat{P}_1, q_1)::\dots::(\hat{P}_n, q_n) \rangle \rightarrow \langle \emptyset, (\hat{P}_1, q_1)::\dots::(\hat{P}_n, q_n) \rangle}
\end{array}$$

Fig. 5: Abstract semantics of the Move borrow checker. Note the absence of interesting premises for most rules—the analysis only performs checks on write instructions (**MoveLoc**, **StoreLoc**, **WriteRef**) or procedure-related instructions (**Call** and **Ret**) that might create a dangling reference.

The order relation on abstract paths is defined by $\langle r, p \rangle \leq \langle r, p' \rangle$ if either $p = p'$ or $p = q*$ and $p' = qq'$ for some relative path q' . We say $\hat{p} = \langle r, p \rangle$ is a *definite path* if p does not end with $*$.

We let $\text{immut} \sqsubset \text{mut}$ to define the partial order $\langle \hat{P}, r \rangle \sqsubseteq \langle \hat{P}', r' \rangle$ if (i) for every $p \in \hat{P}$ there is some $q \in \hat{P}'$ with $p \geq q$ and (ii) $r \sqsubseteq r'$.

An *abstract state* assigns abstract values to local variables and stack positions:

$$\begin{aligned} \hat{L} \in \text{AbsLocals} &= \text{Var} \rightarrow \text{AbsValue} \\ \hat{S} \in \text{AbsStack} &= [\text{AbsValue}] \\ \langle \hat{L}, \hat{S} \rangle \in \text{AbsState} &= \text{AbsLocals} \times \text{AbsStack} \end{aligned}$$

We write $\hat{L}_1(v)$ for the first component of $\hat{L}(v)$, which is a set of abstract paths, and similarly $\hat{L}_2(v)$ for the reference qualifier. Intuitively, if $\langle r, p \rangle \in \hat{L}_1(v)$, then one possible concrete value for v is the value reached by applying field selections p to a concrete value associated with r .

We define an ordering on abstract states that have matching sets of local variables and stack positions by

$$\langle \hat{L}, \hat{S} \rangle \sqsubseteq \langle \hat{L}', \hat{S}' \rangle \text{ if } \begin{cases} \hat{L}(v) \sqsubseteq \hat{L}'(v) \text{ all } v \in \text{dom}(\hat{L}) = \text{dom}(\hat{L}') \\ \hat{S}(i) \sqsubseteq \hat{S}'(i) \text{ all } i \in \text{dom}(\hat{S}) = \text{dom}(\hat{S}') \end{cases}$$

Auxiliary Functions The static analysis rules use the following auxiliary function to obtain the set of paths represented by an abstract state:

$$\text{paths}(\hat{L}, \hat{S}) = \bigcup_{(\hat{A}_j, r_j) \in \text{range}(\hat{L})} \hat{A}_j \cup \bigcup_{(\hat{A}_i, r_i) \in \hat{S}} \hat{A}_i$$

The following predicate checks whether mutable references in $\hat{P}_1$ point to regions of memory that are disjoint from the regions of memory pointed to by all memory in $\hat{P}_2$. This predicate is used in the **Call** rule and **Ret** rule to ensure that formals and return types with a **mut** type annotation “own” the memory they point to. In addition, it is used by the **WriteRef** rule to ensure that a write will not create a dangling reference.

$$\text{disj}(\hat{P}_1, \hat{P}_2) = \neg \exists q. \exists \hat{p}_1 \in \hat{P}_1. \exists \hat{p}_2 \in \hat{P}_2. (\hat{p}_1 \leq q \wedge \hat{p}_2 \leq q)$$

In manipulating paths, we will also need the following concatenation operation that does not add any field names following $*$:

$$p \dagger q = \begin{cases} p & \text{if } p = p'* \\ pq & \text{otherwise} \end{cases}$$

For any set $\hat{P}$ of paths we similarly define

$$\hat{P} \dagger * = \{(r, p \dagger *) \mid (r, p) \in \hat{P}\}$$

4.2 Abstract semantics

Set-based borrow-checker rules are given in Figure 5; we define: $\mathcal{P} \vdash \hat{s} \rightarrow \hat{s}'$ for abstract states $\hat{s}, \hat{s}'$ accordingly. We briefly explain each of the nontrivial rules.

Intuitively., the **MoveLoc** $\langle x \rangle$ and **StoreLoc** $\langle x \rangle$ rules check that the variable x being modified is not a prefix of any path stored in the abstract stack or locals. Similarly, the **WriteRef** checks that none of the abstract paths being written are a prefix of paths stored elsewhere in the abstract stack or locals. The **BorrowField** $\langle f \rangle$ rule extends each abstract path that has a known suffix with the field f .

The **Call** rule is the only rule that introduces abstraction. It performs the following steps:

- Ensures that the callee inputs bound to mutable reference types own the memory they point to
- Creates a set $\hat{P}_i$ that adds **UnkSuffix** for the i th input value passed to the callee. This reflects the fact that the callee can either directly return an input reference value or a value borrowed from an input reference (and the caller cannot not know which).
- For each reference value returned by the procedure, the rule returns a join over some (or all) of these values. If the callee returns a mutable reference, the rule returns only the mutable reference input values $\hat{P}_{mut}$ (since a mutable reference cannot be created from an immutable reference). If the callee returns an immutable references, the rule returns all input values $\hat{P}_{all}$.

Similarly, the **Ret** rule first checks that return values bound to mutable reference types own the memory they point to. Next, it ensures that only values borrowed from inputs to the procedure are returned (i.e., not values borrowed from globals or locals). Finally, the rule checks that the mutability of each returned reference value matches the type signature of the function.

4.3 Soundness

Intuitively, an abstract state $\langle \hat{L}, \hat{S} \rangle$ associated with any program point is intended to characterize the references that occur in the top frame of a concrete state $\langle \langle \rho, \ell, L, S \rangle :: P, M \rangle$. If $L(v) = \mathbf{ref}\langle c, p \rangle$ is a reference occurring in $\langle \rho, \ell, L, S \rangle$, then the concrete path $\langle c, p \rangle$ must be represented by the some abstract path in $\hat{L}(v)_1$ and similarly for any reference in S .

Because abstract values depend on an abstract reference, the correspondence between abstract and concrete states requires a mapping from abstract references to concrete references. To simplify notation in the following definition, we define the *reference map* R for $\langle \hat{L}, \hat{S} \rangle$ and $\langle \langle \rho, \ell, L, S \rangle :: P, M \rangle$ mapping each abstract location $\hat{c}$ in any abstract reference $\mathbf{ref}\langle \hat{c}, p \rangle$ that occurs in $\langle \hat{L}, \hat{S} \rangle$ to a concrete reference $R(\hat{c}) \in \text{dom}(M)$. Specifically,

$R(rp_i) =$ the reference $\mathbf{ref}\langle \hat{c}, p \rangle$ passed to the current procedure invocation as argument x_i ,

$R(rx_i) =$ the reference $\mathbf{ref}\langle \hat{x}_c, [] \rangle$ based on the location x_c for local variable x_i in the current invocation

This gives us a concrete interpretation $R(r)[p]$ for each abstract reference, based on extending the path component of $R(r)$ with path p , as shown in the definition below. While the value of a local variable may change as the procedure executes, the reference that was passed as an initial value remains in the calling frame that is part of the execution sequence of the program.

A concrete state $\langle \langle \rho, \ell, L, S \rangle :: P, M \rangle$ *respects* abstract state $\langle \hat{L}, \hat{S} \rangle$ if:

- (i) For every local variable v in L ,
 - (a) if $L(v) = c$ is a location, then $\hat{L}(v) = (\emptyset, \mathbf{immut})$.
 - (b) if $L(v) = \mathbf{ref}\langle c, p \rangle$ is a reference, then $\langle c, p \rangle \geq \langle c, p_1 :: p_2 \rangle$ for $R(r) = \mathbf{ref}\langle c, p_1 \rangle$ and $\langle r, p_2 \rangle \in \hat{L}(v)$.
- (ii) For every stack position $i \leq \text{len}(S)$,
 - (a) if $S(i) = v$ is a value, then $\hat{S}(i) = (\emptyset, \mathbf{immut})$,
 - (b) if $S(i) = \mathbf{ref}\langle c, p \rangle$ is a reference, then $\langle c, p \rangle \geq \langle c, p_1 :: p_2 \rangle$ for $R(r) = \mathbf{ref}\langle c, p_1 \rangle$ and $\langle r, p_2 \rangle \in \hat{S}(i)$.

Lemma 1. *If concrete state $s = \langle \langle \rho, \ell, L, S \rangle :: P, M \rangle$ respects $\langle \hat{L}, \hat{S} \rangle$ and $\langle \hat{L}', \hat{S}' \rangle \sqsubseteq \langle \hat{L}, \hat{S} \rangle$, then s respects $\langle \hat{L}', \hat{S}' \rangle$.*

A key soundness property is that in any execution, all concrete reachability through references in the concrete state is accounted for in the reachability reflected in the abstract state for that program point. We prove this by first establishing a one-step form of soundness and then prove soundness for any program execution by induction on execution steps. Absence of memory leaks and dangling pointers are then proved by additional induction. Lemmas for single-step soundness and proofs of all lemmas and theorems are in the appendix.

Theorem 1. *If there is an execution sequence for well-typed program $\mathcal{P}$ leading to state $s = \langle \langle \rho, \ell, L, S \rangle :: P, M \rangle$, then concrete state s respects the abstract state $\mathcal{P}[\rho].T[\ell]$ given by the borrow checker annotation at this program point.*

No Memory Leaks In any execution, every local variable bound to a location refers to a distinct location, and locations are not leaked, *i.e.*, the domain of memory includes only locations named in the program. The first of these two conditions is used to prove the second. We state the two conditions more precisely using an auxiliary definition: For any state $s = \langle \langle \rho_1, \ell_1, L_1, S_1 \rangle :: \dots :: \langle \rho_k, \ell_k, L_k, S_k \rangle, M \rangle$ we let $s(i, v) = L_i(v)$ and $s(i, j) = S_i(j)$.

Property $U(s)$: if $s(i, v) = c$ and $s(j, v') = c'$ for either $i \neq j$ or distinct variables $v \neq v'$ then $c \neq c'$.

Property $L(s)$: if $s = \langle P, M \rangle$ and $c \in \text{dom}(M)$ then $c = s(i, v)$ for some integer i less than or equal to the height of stack P and some variable v .

No Dangling References Every reference stems from a location that is the value of some variable in the program, and every location reachable from the program is given a value in memory. The first of these two properties is again used to prove the second. We define both properties more precisely as follows:

Property $R(s)$: if reference $\mathbf{ref}\langle c, p \rangle$ that is the value $s(i, v)$ of some local variable or is in some position $s(i, j)$ on an operand stack, then c is the value $s(i', v')$ of some local variable.

Property $D(s)$: if location is the value $s(i, v)$ of some variable v , then c is in domain of M .

Theorem 2. *Let program $\mathcal{P}$ be well-typed. If concrete state s is reached in the execution of program $\mathcal{P}$ from any empty starting state, then $U(s)$ and $L(s)$ and $D(s)$ and $R(s)$.*

Theorem 1, soundness of the reachability analysis, and Theorem 2, the absence of memory leaks or dangling pointers, are both proved by induction on the length of program execution sequences.

4.4 Global Memory

We have formalized a borrow analysis that operates on a subset of Move with references to local variables on the call stack. In this section, we informally describe how to amend this model to support global storage. The analysis extensions are straightforward additions to the access path set domain that do not require changes to the existing rules.

We model global storage as an extra component G in the concrete program state $\langle P, S, M, G \rangle$, where G maps a type $t \in \text{ValType}$ and an address $a \in \text{Addr}$ to a location $c \in \text{Loc}$. The extended borrow checker uses the **acquires** annotation described in §2.4 to abstract G in the global access instructions described in Section 2.4. We treat each type t in the **acquires** list of a procedure ρ as an extra local variable l_t in the intraprocedural borrow checker rules. The borrow checker ensures that if there is a concrete reference to a value published at $\langle t, a \rangle$ for any address a , then l_t holds an abstract path that concretizes to this reference. This guarantee is achieved by treating **BorrowGlobal** $\langle t \rangle$ similar to **BorrowLoc** $\langle x \rangle$, which allows us to handle **MoveTo** $\langle t \rangle$ and **MoveFrom** $\langle t \rangle$ much like **StoreLoc** $\langle x \rangle$ and **MoveLoc** $\langle x \rangle$ (respectively).

The borrow checker must perform one additional check: at a **MoveFrom** $\langle t \rangle$, **BorrowGlobal** $\langle t \rangle$, or a call to a procedure that has t in its **acquires** list, there must be no outgoing edges from l_t in the borrow graph. Additionally, a separate **acquires** static analysis checks that a procedure with any such instructions has an **acquires** annotation. Together, these ensure that global reference instructions cannot create a dangling reference to global memory.

5 Related Work

Borrow-Based Static Analyses Rust [7] also uses a borrow-based static analysis to prevent dangling references. Rust’s analysis provides reference safety at the

source level, whereas Move provides this guarantee directly for its executable representation via bytecode verification. The analyses support different language features (e.g., Rust allows references in records, Move allows mutable references to global state) and require different annotations (e.g., Rust has reference *lifetime* annotations, Move has procedure *acquires* annotations). We prove that Move’s analysis ensures both absence of dangling references and leak freedom. Reference safety has been proven for Rust [5], but we are not aware of a proof for leak freedom.

There are also differences in the analysis mechanics. There are two descriptions of the Rust borrow checker: one that abstracts *reference lifetimes* and ensures that the lifetimes of related references are properly nested [20] (formalized in [5, 12]), and another that abstracts the relationship between each value and the *set of loans* [8] involving the value and prevents accesses to loaned values (formalized in [2, 23]).

Move’s analysis is more like the second approach, but differs by abstracting the set of access paths each reference value might hold, which preserves valuable structural information about loans and values. This allows simpler handling of features like *reborrowing* (the Rust term for creating a copy of a unique reference), which requires Rust formalizations [2, 23] to maintain additional state, but is the same as a normal borrow for Move. On the other hand, Rust’s reference lifetime annotations allow programmers to precisely specify the relationship between input and output parameters, but Move does not allow this.

Finally, the Move borrow checker is more permissive, as the following (safe) example shows:

```
let mut x = vec![1]; // in Move, this would be written let x = vector[1];
let z = &x;
let y = &mut x;
return *z
```

The Rust borrow checker will reject this code at the `&mut x`—it will not allow a mutable reference to `x` because it opens to the door to creating a dangling reference in `z` via a subsequent mutation. However, the Move borrow checker will allow the code above and (more generally) will only reject code that actually performs a subsequent mutation that might create a dangling reference. Creating a new reference can trigger an error in Rust, but in Move the borrow checker only flags writes or call-related instructions (see Figure 5).

Low-Level Enforcement of Memory Safety Move ensures reference safety for its executable representation without trusting a compiler using the approach of the JVM [3, 6] and CLR [9]: lightweight analysis run in a bytecode verifier. Other approaches to certifying low-level memory safety include typed assembly [10], proof-carrying code [11], and *capability machines*, specialized hardware with a memory-safe instruction set [15, 16, 25]. Each of these approaches has merits in its targeted application domains. We chose a bytecode language with a verifier to accommodate mutable, persistent global storage, gas metering, resource types [13], and other features required for Move’s use-cases.

Bibliography

- [1] Amsden, Z., Arora, R., Bano, S., Baudet, M., Blackshear, S., Bothra, A., Cabrera, G., Catalini, C., Chalkias, K., Cheng, E., Ching, A., Chursin, A., Danezis, G., Giacomo, G.D., Dill, D.L., Ding, H., Doudchenko, N., Gao, V., Gao, Z., Garillot, F., Gorven, M., Hayes, P.J., Hou, J.M., Hu, Y., Hurley, K., Lewi, K., Li, C., Li, Z., Malkhi, D., Margulis, S., Maurer, B., Mohassel, P., de Naurois, L.J., Nikolaenko, V., Nowacki, T., Orlov, O., Perelman, D., Pott, A., Proctor, B.A., Qadeer, S., Russi, D., Schwab, B., Sezer, S., Sonnino, A., Venter, H.A.J., Wei, L., Wernerfelt, N., Williams, B.K., Wu, Q., Yan, X., Zakian, T.A.K., Zhou, R.: The libra blockchain. <https://diem-developers-components.netlify.app/papers/the-diem-blockchain/2020-05-26.pdf> (2019)
- [2] Astrauskas, V., Muller, P., Poli, F., Summers, A.J.: Leveraging rust types for modular specification and verification. *Proc. ACM Program. Lang.* **3**(OOPSLA), 147:1–147:30 (2019)
- [3] Freund, S.N., Mitchell, J.C.: A type system for the java bytecode language and verifier. *J. Autom. Reason.* **30**(3-4), 271–321 (2003)
- [4] Jones, N.D., Muchnick, S.S.: Flow analysis and optimization of LISP-like structures. In: *POPL* (1979)
- [5] Jung, R., Jourdan, J., Krebbers, R., Dreyer, D.: Rustbelt: securing the foundations of the rust programming language. *PACMPL* **2**(POPL), 66:1–66:34 (2018)
- [6] Lindholm, T., Yellin, F.: *The Java Virtual Machine Specification*. Addison-Wesley (1997)
- [7] Matsakis, N.D., Klock, II, F.S.: The rust language. *Ada Lett.* **34**(3), 103–104 (Oct 2014)
- [8] Matsakis, N.: An alias-based formulation of the borrow checker. <http://smallcultfollowing.com/babysteps/blog/2018/04/27/an-alias-based-formulation-of-the-borrow-checker/> (2018)
- [9] Meijer, E., Wa, R., Gough, J.: Technical overview of the common language runtime (2000)
- [10] Morrisett, J.G., Crary, K., Glew, N., Walker, D.: Stack-based typed assembly language. *J. Funct. Program.* **13**(5), 957–959 (2003)
- [11] Necula, G.C.: Proof-carrying code. In: van Tilborg, H.C.A., Jajodia, S. (eds.) *Encyclopedia of Cryptography and Security*, 2nd Ed, pp. 984–986. Springer (2011)
- [12] Reed, E.: *Patina: A formalization of the rust programming language*. Tech. rep., University of Washington (2015)
- [13] for double-blind review, W.: Move: A language with programmable resources (2019)
- [14] for double-blind review, W.: The move prover. In: Lahiri, S.K., Wang, C. (eds.) *CAV. Lecture Notes in Computer Science*, vol. 12224, pp. 137–150. Springer (2020)

- [15] Skorstengaard, L., Devriese, D., Birkedal, L.: Stktokens: enforcing well-bracketed control flow and stack encapsulation using linear capabilities. *Proc. ACM Program. Lang.* **3**(POPL), 19:1–19:28 (2019)
- [16] Skorstengaard, L., Devriese, D., Birkedal, L.: Reasoning about a machine with local capabilities: Provably safe stack and return pointer management. *ACM Trans. Program. Lang. Syst.* **42**(1), 5:1–5:53 (2020)
- [17] Szabo, N.: Formalizing and securing relationships on public networks. *First Monday* **2**(9) (1997), <https://ojphi.org/ojs/index.php/fm/article/view/548>
- [18] Team, L.: 0l network. <https://0l.network/technology/>
- [19] Team, A.: The aptos blockchain: Safe, scalable, and upgradeable web3 infrastructure. <https://aptos.dev/assets/files/Aptos-Whitepaper-47099b4b907b432f81fc0effd34f3b6a.pdf>
- [20] core team, R.: Rfcs: 2094-nll. <https://github.com/rust-lang/rfcs/blob/master/text/2094-nll.md> (2017)
- [21] Team, S.: Introduction to starcoin. https://starcoin.org/downloads/Starcoin_Whitepaper.pdf
- [22] Team, T.M.L.: The sui smart contracts platform. <https://docs.sui.io/paper/sui.pdf>
- [23] Weiss, A., Patterson, D., Matsakis, N.D., Ahmed, A.: Oxide: The essence of rust (2019)
- [24] Wood, G.: Ethereum: A secure decentralised generalised transaction ledger (2014), <https://ethereum.github.io/yellowpaper/paper.pdf>
- [25] Woodruff, J.: CHERI: a RISC capability machine for practical memory safety. Ph.D. thesis, University of Cambridge, UK (2014)

A Proof

A.1 Soundness of static borrow analysis

We prove soundness of the borrow checker by showing inductive invariants between concrete and abstract states. The most interesting cases of each proof are described below.

Well-typed programs For each procedure ρ , we assume the borrow checker provides an abstract local state $\mathcal{P}[\rho].T[i]$ for each instruction offset $i \in 0..len(\mathcal{P}[\rho].C)$. These abstract local states must satisfy the initial state, control flow, and return conditions below for the program to be well-typed.

Initial state condition. We define an initial state $\hat{\sigma}_{init} = \langle \hat{L}_{init}, \hat{S}_{init} \rangle$ for procedure ρ and say that a type annotation of ρ meets the initial state condition if $\hat{\sigma}_{init} \sqsubseteq \mathcal{P}[\rho].T[0]$. Recall that local variables $v_1, \dots, v_k$ of procedure ρ are initialized to the parameters passed by calling the procedure. We take $\hat{S}_{init} = \emptyset$ and let $\hat{L}_{init}(v_i) = \langle \{ \langle rp_i, \epsilon \rangle \}, q_i \rangle$ for each formal parameter v_i with qualifier q_i in $\mathcal{P}[\rho].formals$, where $rp_1, \dots, rp_k$ are the abstract references intended to represent the actual references passed by the call.

$$\boxed{\mathcal{P} \vdash \langle L, S, M \rangle \rightarrow \langle L', S', M' \rangle}$$

$$\begin{array}{c}
\frac{L(x) = c}{\text{MoveLoc } \langle x \rangle \vdash \langle L, S, M \rangle \rightarrow \langle L \setminus x, M(c) :: S, M \setminus c \rangle} \text{Val} \quad \frac{L(x) = \mathbf{ref}\langle c, p \rangle}{\text{MoveLoc } \langle x \rangle \vdash \langle L, S, M \rangle \rightarrow \langle L \setminus x, \mathbf{ref}\langle c, p \rangle :: S, M \rangle} \text{Re} \\
\\
\frac{L(x) = c}{\text{CopyLoc } \langle x \rangle \vdash \langle L, S, M \rangle \rightarrow \langle L, M(c) :: S, M \rangle} \text{Val} \quad \frac{L(x) = \mathbf{ref}\langle c, p \rangle}{\text{CopyLoc } \langle x \rangle \vdash \langle L, S, M \rangle \rightarrow \langle L, \mathbf{ref}\langle c, p \rangle :: S, M \rangle} \text{Ref} \\
\\
\frac{L(x) = c}{\text{StoreLoc } \langle x \rangle \vdash \langle L, v :: S, M \rangle \rightarrow \langle L, S, M[c \mapsto v] \rangle} \text{Val} \quad \frac{x \notin \text{dom}(L)}{\text{StoreLoc } \langle x \rangle \vdash \langle L, v :: S, M \rangle \rightarrow \langle L[x \mapsto c_x], S, M[c_x \mapsto v] \rangle} \\
\\
\frac{}{\text{StoreLoc } \langle x \rangle \vdash \langle L, r :: S, M \rangle \rightarrow \langle L[x \mapsto r], S, M \rangle} \text{Ref} \quad \frac{}{\text{BorrowLoc } \langle x \rangle \vdash \langle L, S, M \rangle \rightarrow \langle L, \mathbf{ref}\langle L(x), [] \rangle :: S, M \rangle} \\
\\
\frac{r = \mathbf{ref}\langle c, p \rangle}{\text{BorrowField } \langle f \rangle \vdash \langle L, r :: S, M \rangle \rightarrow \langle L, \mathbf{ref}\langle c, p :: f \rangle :: S, M \rangle} \\
\text{FreezeRef } \vdash \langle L, r :: S, M \rangle \rightarrow \langle L, r :: S, M \rangle \\
\\
\frac{}{\text{ReadRef } \vdash \langle L, \mathbf{ref}\langle c, p \rangle :: S, M \rangle \rightarrow \langle L, M(c)[p] :: S, M \rangle} \\
\frac{r = \mathbf{ref}\langle c, p \rangle \quad v = M(c)}{\text{WriteRef } \vdash \langle L, v' :: r :: S, M \rangle \rightarrow \langle L, S, M[c \mapsto v[p := v']] \rangle} \\
\\
\frac{t = \{(f_i, -) \mid 1 \leq i \leq n\} \quad v = \{(f_i, v_i) \mid 1 \leq i \leq n\}}{\text{Pack } \langle t \rangle \vdash \langle L, v_1 :: \dots :: v_n :: S, M \rangle \rightarrow \langle L, v :: S, M \rangle} \quad \frac{v = \{(f_i, v_i) \mid 1 \leq i \leq n\}}{\text{Unpack } \vdash \langle L, v :: S, M \rangle \rightarrow \langle L, v_1 :: \dots :: v_n :: S, M \rangle} \\
\\
\frac{}{\text{Pop } \vdash \langle L, :: S, M \rangle \rightarrow \langle L, S, M \rangle} \quad \frac{}{\text{Op } \vdash \langle L, v_1 :: \dots :: v_n :: S, M \rangle \rightarrow \langle L, \mathbf{Op}(v_1, \dots, v_n) :: S, M \rangle} \\
\\
\frac{\mathcal{P}[\rho].C[\ell] \vdash \langle L, S, M \rangle \rightarrow \langle L', S', M' \rangle}{\mathcal{P} \vdash \langle (\rho, \ell, L, S) :: P, M \rangle \rightarrow \langle (\rho, \ell + 1, L', S') :: P, M' \rangle} \text{Local} \quad \frac{z = (\rho, \ell, L, \bar{u} :: S) \quad \mathcal{P}[\rho].C[\ell] = \mathbf{Call}\langle \rho' \rangle}{\mathcal{P} \vdash \langle z :: P, M \rangle \rightarrow \langle (\rho', 0, \text{rev}(\bar{u}), \varepsilon) :: (\rho, \ell + 1, L, S) :: P, M \rangle} \\
\\
\frac{z = (\rho, \ell, L, []) \quad \mathcal{P}[\rho].C[\ell] = \mathbf{Ret} \quad M' = M - \text{img}(L)}{\mathcal{P} \vdash \langle z, M \rangle \rightarrow \langle [], M' \rangle} \text{Emp} \\
\frac{z = (\rho, \ell, L, S) \quad \mathcal{P}[\rho].C[\ell] = \mathbf{Ret} \quad M' = M - \text{img}(L)}{\mathcal{P} \vdash \langle z :: (\rho', \ell', L', S') :: P, M \rangle \rightarrow \langle (\rho', \ell', L', S :: S') :: P, M' \rangle} \text{Ret} \\
\\
\frac{\mathcal{P}[\rho].C[\ell] = \mathbf{Branch} \langle \ell_1, \ell_2 \rangle \quad \ell' = \text{ite}(v, \ell_1, \ell_2)}{\mathcal{P} \vdash \langle (\rho, \ell, L, v :: S) :: P, M \rangle \rightarrow \langle (\rho, \ell', L, S) :: P, M \rangle}
\end{array}$$

Fig. 6: Small-step operational semantics for Move bytecode.

Control-flow condition. The control flow condition uses $Next(\rho)[i]$ to refer to the set of next instructions after executing instruction i , specifically,

$$Next(\rho)[i] = \begin{cases} \{\}, & \text{if } \mathcal{P}[\rho].C[i] = \mathbf{Ret} \\ \{\ell_1, \ell_2\}, & \text{if } \mathcal{P}[\rho].C[i] = \mathbf{Branch} \langle \ell_1, \ell_2 \rangle \\ \{\ell + 1\}, & \text{otherwise} \end{cases}$$

A type annotation of procedure ρ meets the control-flow condition if, for all $i \in dom(\mathcal{P}[\rho].C)$, there is an abstract state $\hat{\sigma}$ such that $\rho, \mathcal{P}[\rho].C[i] \vdash \mathcal{P}[\rho].T[i] \rightarrow \hat{\sigma}$ and, for all $j \in Next(\rho)[i]$, $\hat{\sigma} \sqsubseteq \mathcal{P}[\rho].T[j]$.

In other words, any conditions expressed in the abstract state propagation rules must be met and the abstract state produced by applying the propagation rules must be $\sqsubseteq$ the annotation of any next program state. We do not require equality because there may be more than one control path for reaching the next instruction and the borrow checking algorithm may therefore select some abstract state that is $\sqsupseteq$ all of them.

Return condition. If $\mathcal{P}[\rho].C[i] = \mathbf{Ret}$, then $\rho, \mathcal{P}[\rho].C[i] \vdash \mathcal{P}[\rho].T[i] \rightarrow \hat{\sigma}$ with $\hat{\sigma} \sqsubseteq \mathcal{P}[\rho].O$.

A program $\mathcal{P}$ is *well-typed* if, for all $\rho \in dom(\mathcal{P})$, the abstract local $\mathcal{P}[\rho].T[i]$ for each offset $i \in 0..len(\mathcal{P}[\rho].C)$ is well formed, the initial state condition holds for ρ , the control flow condition holds for all instruction points $i \in 0..len(\mathcal{P}[\rho].C)$, and the return condition holds.

Lemma 2. (*One-step soundness for local instructions*) Suppose program $\mathcal{P}$ is well-typed, concrete state $s = \langle \langle \rho, \ell, L, S \rangle :: P, M \rangle$ respects abstract state $\langle \hat{L}, \hat{S} \rangle$, instruction $\rho[\ell]$ is not call or return, and $\mathcal{P} \vdash s \rightarrow s'$ in the concrete semantics. Then $\mathcal{P} \vdash \langle \hat{L}, \hat{S} \rangle \rightarrow \langle \hat{L}', \hat{S}' \rangle$ in the abstract semantics and s' respects $\langle \hat{L}', \hat{S}' \rangle$.

In proving soundness for call and return, we use a condition relating the references passed to a procedure and those returned. Specifically, if $\mathbf{ref}\langle c_1, p_1 \rangle, \dots, \mathbf{ref}\langle c_k, p_k \rangle$ is a list that includes all the references passed to a procedure, then v is a *possible return value* if either (i) v is a value, or (ii) v extends one of the references passed as an argument, i.e., $v = \langle c, p \rangle = \langle c_i, p_i :: p' \rangle$ for $1 \leq i \leq k$ and some path p' .

Lemma 3. (*One-step soundness for call*) Let program $\mathcal{P}$ be well-typed. Suppose concrete state $s = \langle \langle \rho_1, \ell_1, L_1, \bar{u} :: S_1 \rangle :: P, M \rangle$ respects abstract state $\langle \hat{L}_1, (\hat{P}, q) :: \hat{S}_1 \rangle$. Assume instruction $\rho_1[\ell_1]$ is a call to a procedure ρ_0 with m parameters, $\bar{u}$ is correspondingly a list of m stack entries, and $(\hat{P}, q) = (\hat{P}_1, q_1) :: \dots :: (\hat{P}_m, q_m)$. If

$$\mathcal{P} \vdash s \rightarrow s' = \langle \langle \rho_0, 0, rev(\bar{u}), \varepsilon \rangle :: \langle \rho_1, \ell_1 + 1, L_1, S_1 \rangle :: P, M \rangle$$

then s' respects $\hat{\sigma}_{init} = \langle \hat{L}_{init}, \hat{S}_{init} \rangle$, the initial abstract state for procedure ρ_0 given in the initial state condition for type correctness. In addition, for any possible return values $\bar{w}$ the state

$$s'' = \langle \langle \rho_1, \ell_1 + 1, L_1, \bar{w} :: S_1 \rangle :: P, M \rangle$$

obtained by placing return values on top of the post-call operand stack respects

$$\langle \hat{L}_1, \hat{A}_1 :: \dots :: \hat{A}_n :: \hat{S}_1 \rangle$$

where $\hat{A}_1, \dots, \hat{A}_n$ are as in the abstract semantic rule for **Call** $\langle \rho_1 \rangle$.

Lemma 4. (One-step soundness for return) Suppose $s_0, \dots, s_n$ is an execution sequence for well-typed program $\mathcal{P}$,

$$s_n = \langle (\rho_0, \ell_0, L_0, S_0) :: (\rho_1, \ell_1, L_1, S_1) :: P, M \rangle$$

respects abstract state $\langle \hat{L}, \hat{S} \rangle$, instruction $\rho_0[\ell_0]$ is return, and $\mathcal{P} \vdash s_n \rightarrow s_{n+1} = \langle (\rho_1, \ell_1, L_1, S_0 :: S_1) :: P, M' \rangle$ in the concrete semantics. Let s_m be the unique prior state, $m < n$, whose next instruction calls ρ_0 and pushes the current top frame $(\rho_0, \ell_0, \dots, \dots)$ onto the call stack and let $\langle \hat{L}_m, \hat{S}_m \rangle = \mathcal{P}[\rho].T[\ell]$ be the abstract state corresponding to s_m . Then S_0 is a list of possible return values for ρ at state s_m .

Proof (Proof of Lemma 2). Suppose program $\mathcal{P}$ is well-typed, concrete state $s = \langle \langle \rho, \ell, L, S \rangle :: P, M \rangle$ respects abstract states $\langle \hat{L}_1, \hat{S}_1 \rangle$, instruction $\rho[\ell]$ is not call or return, and $\mathcal{P} \vdash s \rightarrow s'$ in the concrete semantics. We must show that $\mathcal{P} \vdash \langle \hat{L}_1, \hat{S}_1 \rangle \rightarrow \langle \hat{L}'_1, \hat{S}'_1 \rangle$ in the local abstract semantics such that s' respects $\langle \hat{L}'_1, \hat{S}'_1 \rangle$.

The proof is by case analysis on local instructions. We begin with **MoveLoc** $\langle x \rangle$.

There are two cases, depending on whether $L(x)$ is a location or reference. If $L(x) = c$ is a location, then the instruction removes the local variable x from the stack frame, pushes the value $v = M(c) = M(L(x))$ onto the operand stack, and removes c from memory. This maps concrete state $s = \langle \langle \rho, \ell, L, S \rangle :: P, M \rangle$ to $s' = \langle \langle \rho, \ell, L \setminus x, v :: S \rangle :: P, M \setminus c \rangle$. The abstract semantics check that $\text{disj}(\{\hat{L}(x)\}, \text{paths}(\hat{S}, \hat{L} \setminus x))$ and maps $\langle \hat{L}_1, \hat{S}_1 \rangle$ to $\langle \hat{L}'_1, \hat{S}'_1 \rangle = \langle \hat{L}_1 \setminus x, (\emptyset, \text{immut}) :: \hat{S}_1 \rangle$. We must show that s' respects $\langle \hat{L}'_1, \hat{S}'_1 \rangle :: \dots :: \langle \hat{L}_k, \hat{S}_k \rangle$.

For every local variable x' in $\hat{L} \setminus x$, the conditions in the definition of *respects* hold because the concrete and abstract semantics associated with the variable x' are unchanged. For the new top value v on the stack, the abstract semantics now have $(\emptyset, \text{immut})$ on top of the abstract stack. Thus the lemma holds for this case.

Alternatively, if $u = L(x) = \text{ref} \langle c, p \rangle$ is a reference, then the instruction removes the local variable x from the stack frame and pushes the reference u onto the operand stack. This maps concrete state $s = \langle \langle \rho, \ell, L, S \rangle :: P, M \rangle$ to $s' = \langle \langle \rho, \ell, L \setminus x, u :: S \rangle :: P, M \rangle$. The abstract semantics map $\langle \hat{L}_1, \hat{S}_1 \rangle$ to $\langle \hat{L}'_1, \hat{S}'_1 \rangle = \langle \hat{L}_1 \setminus x, \hat{L}(x) :: \hat{S}_1 \rangle$. We must show that s' respects $\langle \hat{L}'_1, \hat{S}'_1 \rangle$.

For each local variable x' in $\hat{L} \setminus x$ other than x , and each stack position other than the new top (reference $u = L(x)$), the relevant conditions hold by assumption that s respects $\hat{s}$. We also know by assumption that for local variable x of state s , if $L(x) = u = \mathbf{ref}\langle c, p \rangle$ is a reference, then $\langle c, p \rangle \geq \langle c, p_1 :: p_2 \rangle$ for $R(r) = \mathbf{ref}\langle c, p_1 \rangle$ and $\langle r, p_2 \rangle \in \hat{L}(x)$.

We now consider $S'(0) = u = \mathbf{ref}\langle c, p \rangle$ which was pushed on the top of the operand stack by **MoveLoc** $\langle x \rangle$. We must show that $M'(c).p \geq R(\hat{c}).p_2$ for some $\langle \hat{c}, p_2 \rangle \in \hat{S}'(i)$. But since $\hat{S}'(i) = \hat{L}(x)$, this follows by assumption that this held for $L(x)$ and $\hat{L}(x)$ in the initial state.

The argument for **CopyLoc** $\langle x \rangle$ is similar to **MoveLoc** $\langle x \rangle$.

We now consider **StoreLoc** $\langle x \rangle$. There are two concrete semantic rules, depending on whether the top of the operand stack is a reference or value.

If the concrete local state $\langle L, r :: S, M \rangle$ has a reference $r = \mathbf{ref}\langle c, p \rangle$ on top of the stack, the concrete semantics map $s = \langle \langle \rho, \ell, L, S \rangle :: P, M \rangle$ to $s' = \langle \langle \rho, \ell, L[x \mapsto r], S \rangle :: P, M \rangle$ with local variable x bound to r . The abstract semantics of **StoreLoc** $\langle x \rangle$ maps $\langle \hat{L}_1, \hat{A} :: \hat{S}_1 \rangle$ to $\langle \hat{L}'_1, \hat{S}'_1 \rangle = \langle \hat{L}_1[x \mapsto \hat{A}], \hat{S}_1 \rangle$. We must show that s' respects $\langle \hat{L}'_1, \hat{S}'_1 \rangle$.

With $\hat{L}'(x) = \mathbf{ref}\langle c, p_1 \rangle$, we must show that $\langle c, p \rangle \geq \langle c, p_1 :: p_2 \rangle$ for $R(r) = \mathbf{ref}\langle c, p_1 \rangle$ and $\langle r, p_2 \rangle \in \hat{L}'(x) = \hat{A}$. But this follows immediately from the hypothesis that $\langle c, p \rangle \geq \langle c, p_1 :: p_2 \rangle$ for $R(r) = \mathbf{ref}\langle c, p_1 \rangle$ and $\langle r, p_2 \rangle \in \hat{A}$.

Alternatively, if the concrete initial state $\langle L, v :: S, M \rangle$ has value v on top of the operand stack, there are two subcases, according to whether the variable x is already defined in the current state.

Suppose $L(x) = c$ is a location. The concrete semantics map $s = \langle \langle \rho, \ell, L, v :: S \rangle :: P, M \rangle$ to $s' = \langle \langle \rho, \ell, L, S \rangle :: P, M[c \mapsto v] \rangle$. The abstract semantics maps $\langle \hat{L}_1, \hat{A} :: \hat{S}_1 \rangle$ to $\langle \hat{L}'_1, \hat{S}'_1 \rangle = \langle \hat{L}_1[x \mapsto \hat{A}], \hat{S}_1 \rangle$. We must show that s' respects $\langle \hat{L}'_1, \hat{S}'_1 \rangle$.

By hypothesis, we assume $\hat{A} = (\emptyset, \mathbf{immut})$. We must show that condition (i)(a) of the definition of *respects* holds here. Because $L(x) = c$ as before the instruction, and $\hat{L}(x) = (\emptyset, \mathbf{immut})$, the lemma holds.

The remaining subcase is for concrete initial state $\langle L, v :: S, M \rangle$ with value v on the operand stack, with local variable $x \notin \text{dom}(L)$ not defined in the starting state. The command binds x to the designated location $c_x \notin \text{dom}(M)$ and places v at that location in memory, resulting in state $\langle L', S', M' \rangle = \langle L[x \mapsto c], S, M[c \mapsto v] \rangle$. Because $L(x)$ is a location and $\hat{A} = (\emptyset, \mathbf{immut})$ by hypothesis of the abstract semantic rule, the reasoning for condition (i)(a) of the definition of *respects* is the same as the previous case.

For **WriteRef**, the concrete semantics map $\langle L, v :: \mathbf{ref}\langle c, p \rangle :: S, M \rangle$ to $\langle L, S, M' \rangle$, where $M' = M[c \mapsto v[p := v']]$ and $v = M(c)$. The abstract semantics map abstract local state $\langle \hat{L}, \hat{A} :: (\hat{P}, \mathbf{mut}) :: \hat{S} \rangle$ with $\hat{A} = (\emptyset, \mathbf{immut})$ to $\langle \hat{L}, \hat{S} \rangle$. We assume $\text{disj}(\hat{P}, \text{paths}(\hat{S}, \hat{L}))$ since the program passes the borrow checker.

By hypothesis, we know that for every variable x , if $L(x) = \mathbf{ref}\langle c', p' \rangle$ is a reference, then $\langle c', p' \rangle \geq \langle c', p_1 :: p_2 \rangle$ for $R(r) = \mathbf{ref}\langle c, p_1 \rangle$ and $\langle r, p_2 \rangle \in \hat{L}(x)$.

Similarly, for every stack position $i \leq \text{len}(S)$, if $S(i) = \mathbf{ref}\langle c', p' \rangle$, then $\langle c', p' \rangle \geq \langle c, p_1 :: p_2 \rangle$ for $R(r) = \mathbf{ref}\langle c, p_1 \rangle$ and $\langle r, p_2 \rangle \in \hat{S}(i)$. We must show that both remain true after the instruction. However, this is evident because only memory has changed, and the conditions required to prove the lemma do not depend on memory.

We omit the remaining cases. $\square$

Proof (Proof of Lemma 3). We assume program $\mathcal{P}$ is well-typed, concrete state $s = \langle \langle \rho_1, \ell_1, L_1, \bar{u} :: S_1 \rangle :: P, M \rangle$ respects abstract state $\langle \hat{L}_1, (\hat{P}, q) :: \hat{S}_1 \rangle$, instruction $\rho_1[\ell_1]$ calls procedure ρ_0 with m parameters, $\bar{u} = u_1 :: \dots :: u_m$ is correspondingly a list of m stack entries, and $(\hat{P}, q) = (\hat{P}_1, q_1) :: \dots :: (\hat{P}_m, q_m)$ is a list of m abstract values. If the call results in state transition

$$\mathcal{P} \vdash s \rightarrow s' = \langle \langle \rho_0, 0, \text{rev}(\bar{u}), \varepsilon \rangle :: \langle \rho_1, \ell_1 + 1, L_1, S_1 \rangle :: P, M \rangle$$

then we must show that s' respects $\hat{\sigma}_{init} = \langle \hat{L}_{init}, \hat{S}_{init} \rangle$, the initial abstract state for procedure ρ_0 with $\hat{S}_{init} = \emptyset$ and $\hat{L}_{init}(v_i) = \langle \{ \langle rp_i, \epsilon \rangle \}, q_i \rangle$ for each formal parameter v_i with qualifier q_i in $\mathcal{P}[\rho].formals$, and $rp_1, \dots, rp_k$ the abstract references chosen to represent the actual references passed by the call. Note that $\hat{L}_{init}$ depends only on $\mathcal{P}[\rho].formals$ and is independent of $(\hat{P}, q)$. Since $\text{rev}(\bar{u})$ becomes the local variable assignment in the top frame of the concrete state, we use the notation $L(v_i) = u_i$.

We can see that s' respects $\hat{L}_{init}$ by observing that if $L(v_i) = u_i = \mathbf{ref}\langle c, p \rangle$ is a reference, then $\langle c, p \rangle = R(rp_i) = \mathbf{ref}\langle c, p \rangle$ because R maps abstract reference rp_i to the i th actual parameter value.

To prove the second part of the lemma, we must show that, for any possible return values $\bar{w}$, the state

$$s'' = \langle \langle \rho_1, \ell_1 + 1, L_1, \bar{w} :: S_1 \rangle :: P, M \rangle$$

obtained by placing return values on top of the post-call operand stack respects

$$\langle \hat{L}_1, \hat{A}_1 :: \dots :: \hat{A}_n :: \hat{S}_1 \rangle$$

where $\hat{A}_1, \dots, \hat{A}_n$ are as in the abstract semantic rule for **Call** $\langle \rho_1 \rangle$. Specifically, we let $q_i = \mathbf{mut}$ if $\rho'.ins[i].\tau = \mathbf{MutRef}$ and $q_i = \mathbf{immut}$ otherwise, assume $\text{owned}(\hat{P}_i, q_i, \text{paths}(\hat{S}, \hat{L}) \cup \bigcup_{j=1 \dots m, j \neq i} \hat{P}_j)$ and let $\hat{A}_i = \bigcup_{j=1 \dots m} (\hat{P}_j \dagger *)$ for each

i. All that is required, therefore, is that if $w_i = \mathbf{ref}\langle c, p \rangle$ is a reference, then $\langle c, p \rangle \geq \langle c, p_1 \rangle$ for some $\mathbf{ref}\langle c, p_1 \rangle$ in $\bigcup_{j=1 \dots m} \hat{P}_j$. In other words, by the assumption

of the lemma, it suffices to show that any returned reference must be an extension of a reference passed to the procedure. However, this is the definition of *possible return value*. $\square$

Proof (Proof of Lemma 4). As in the statement of the lemma, we assume that $s_0, \dots, s_n$ is an execution sequence for well-typed program $\mathcal{P}$,

$$s_n = \langle (\rho_0, \ell_0, L_0, S_0) :: (\rho_1, \ell_1, L_1, S_1) :: P, M \rangle$$

respects abstract state $\langle \hat{L}, \hat{S} \rangle$, instruction $\rho_0[\ell_0]$ is return, and $\mathcal{P} \vdash s_n \rightarrow s_{n+1} = \langle (\rho_1, \ell_1, L_1, S_0 :: S_1) :: P, M' \rangle$ in the concrete semantics. Let s_m be the unique prior state, $m < n$, whose next instruction calls ρ_0 and pushes the top frame $(\rho_0, \ell_0, \dots, \dots)$ of s_n onto the call stack and let $\langle \hat{L}_m, \hat{S}_m \rangle = \mathcal{P}[\rho].T[\ell]$ be the abstract state corresponding to the program point of concrete state s_m . We must show that S_0 is a list of *possible return values for ρ* , based on the arguments passed to ρ in the call executed from state s_m .

The proof of the lemma uses the static semantic rule, which includes the condition that

$$\forall i \in 1 \dots n, \forall (r_j, p) \in \hat{P}_i, j < \text{len}(\rho.\text{ins})$$

if $(r, p) \in \hat{P}_i$, then $r \in \{rp_1, \dots, rp_n\}$, where $n = \text{len}(\rho.\text{outs})$. This would directly state that every reference that is return from the procedure depends only on references that are passed as arguments (I think).

Proof (Proof of Theorem 1). Suppose $s_0, \dots, s_n$ is an execution sequence for well-typed program $\mathcal{P}$ that leads to state

$$s_n = \langle (\rho, \ell, L, S) :: P, M \rangle$$

We show by induction on the length, n , of the execution sequence that s_n respects the abstract state $\mathcal{P}[\rho].T[\ell]$ given by the borrow checker annotation at this program point.

If $n = 0$, then s_0 is an initial state with a call to a initial procedure and the lemma holds by definition of initial state.

For the inductive step, assume the theorem holds for all $i < n$, including s_{n-1} . If the instruction leading from state s_{n-1} to s_n is a local instruction, then $s_{n-1} = \langle (\rho, \ell - 1, L', S') :: P, M' \rangle$. By the inductive hypothesis, s_{n-1} respects abstract state $\mathcal{P}[\rho].T[\ell - 1]$. $\square$

A.2 Memory safety and absence of memory leaks

Property $U(s)$: if $s(i, v) = s(j, v') = c$ for any variables v, v' and stack positions i, j , then $i = j$ are the same stack position and $v = v'$ are the same variable.

Property $L(s)$: if $s = \langle P, M \rangle$ and $c \in \text{dom}(M)$ then $c = s(i, v)$ for some integer i less than or equal to the height of stack P and some variable v .

Property $R(s)$: if reference $\text{ref}(c, p)$ that is the value $s(i, v)$ of some local variable or is in some position $s(i, j)$ on an operand stack, then c is the value $s(i', v')$ of some local variable.

Property $D(s)$: if location is the value $s(i, v)$ of some variable v , then c is in domain of M .

Proof (Property U). The only local instruction that changes which location a local variable is bound to does not violate property U because it introduces a new location. Specifically, there are several cases in the concrete semantics of

StoreLoc $\langle x \rangle$. The first is an initial state $\langle L, r::S, M \rangle$ with reference r on top of the stack, In this case, execution leads to state $\langle L', S', M' \rangle = \langle L[x \mapsto r], S, M \rangle$ with local variable x bound to r . This does not affect property U .

The remaining two cases arise from the Val rule, where the initial state $\langle L, v::S, M \rangle$ has value v on top of the operand stack. If $L(x) = c$ is a location, then the command places v in memory in resulting state $\langle L', S', M' \rangle = \langle L, S, M[c \mapsto v] \rangle$.

If the local variable $x \notin \text{dom}(L)$ is not previously defined we choose a fresh location $c \notin \text{dom}(M)$ and the command binds x to c and places v in memory in resulting state $\langle L', S', M' \rangle = \langle L[x \mapsto c], S, M[c \mapsto v] \rangle$. This preserves property U because the location c is not previously any part of the state.

The argument for **Call** $\langle \rho \rangle$ is straightforward because only values and references are passed as arguments. No new location variables are created.

The **Ret** command removes local variables by discarding the top frame of the call stack. This does not create new variables that name locations (only has the potential to remove them), so preserves Property U. $\square$

Proof (Property L). We prove Property L, leak-freedom, by induction on program execution. The base case is an initial call to a procedure, from a state that is leak-free by construction.

For the induction, suppose program $\mathcal{P}$ is well-typed, state $s = \langle \langle \rho, \ell, L, S \rangle :: P, M \rangle$ satisfies Property L, and $\mathcal{P} \vdash s \rightarrow s'$. The proof proceeds by case analysis on the instruction $\rho[\ell]$.

Call $\langle \rho \rangle$ only adds values and references to the call-stack, so does not alter Property L.

Ret removes local variables by removing the top frame from the call-stack. However, the domain of memory is correspondingly reduced by setting the memory M' for state s' to $M' = M - \text{img}(L)$, as is evident in the semantic rules. Therefore, procedure returns preserve Property L. **MoveLoc** $\langle x \rangle$ with $L(x) = c$ similarly removes a local variable from the call stack, and reduces the domain of memory by $M \setminus c$.

For **MoveLoc** $\langle x \rangle$ with $L(x) = \text{ref}\langle c, p \rangle$ the local variable x is removed from the call stack. However, by Property R, , the location c remains the value of some other variable accessible to the program, thus preserving Property L.

StoreLoc $\langle x \rangle$ with $x \notin \text{dom}(L)$ expands the domain of memory by $M[c_x \mapsto v]$, but also adds local variable x bound to c . This property L is preserved.

No other instructions extend the domain of memory or remove local variables from the stack. $\square$

Property R To prove this property, we define the *location reachability property* inductively. Specifically, a concrete state $s = \langle [], M \rangle$ with empty stack satisfies the location reachability property and a concrete state $s = \langle \langle \rho, \ell, L, S \rangle :: P, M \rangle$ with nonempty stack satisfies the location reachability property if

- (i) for any reference $\text{ref}\langle c, p \rangle$ that is the value of a local variable of L or in any position on the operand stack S , the location c is also the value of a local variable in L or some lower frame of the runtime stack P , and

(ii) $\langle P, M \rangle$ satisfies the location reachability property.

Lemma 5 (R). *Suppose program $\mathcal{P}$ is well-typed and $\mathcal{P} \vdash s \rightarrow s'$ by type-correct execution in the concrete semantics. If s satisfies the location reachability property then s' satisfies the location reachability property.*

Proof (Proof of Lemma 5). The proof is by induction on the depth of the call stack. If $s = \langle [], M \rangle$ with empty stack, then no instructions are possible so the lemma holds trivially.

For concrete state $s = \langle \langle \rho, \ell, L, S \rangle :: P, M \rangle$ with nonempty stack, the proof proceeds by case analysis on the instruction $\mathcal{P}[\rho].C[\ell]$, which must be either a local instruction, call or return.

For any local instruction, we suppose that if $L(x') = \mathbf{ref}\langle c, p \rangle$ or $S(i) = \mathbf{ref}\langle c, p \rangle$ for any x' or any i , then the location c is the value of a variable in L or P .

Instruction **MoveLoc** $\langle x \rangle$ may remove a variable from L . We must show when this instruction executes type-correctly, it does not eliminate a location needed by another reference. There are two cases. If $L(x) = c$, then there must be no references in s that depend on this location. However, that is guaranteed by the hypothesis $\text{disj}(\{(r, \epsilon)\}, \text{paths}(\hat{S}, \hat{L} \setminus x))$ of the static semantics rule for **MoveLoc** $\langle x \rangle$. Alternatively, if $L(x) = \mathbf{ref}\langle c, p \rangle$, then by hypothesis the location c is the value of some other variable in L or P . The lemma therefore holds in state s' with variable x eliminated.

Instruction **CopyLoc** $\langle x \rangle$ follows by similar reasoning and **StoreLoc** $\langle x \rangle$ is essentially straightforward.

The local instructions producing a reference in s' that was not in state s are **BorrowLoc** $\langle x \rangle$ and **BorrowField** $\langle f \rangle$. The lemma follows immediately for **BorrowLoc** $\langle x \rangle$ because the location used in the new reference comes from $L(x)$. The lemma holds for **BorrowField** $\langle f \rangle$ because the new reference $\mathbf{ref}\langle c, p :: f \rangle$ uses a location c that occurs on a reference on the stack of state s .

Because **ReadRef** and **WriteRef** are straightforward, it remains to prove the lemma for call and return. For call, the state s' has no new references, and no local variables are removed from the call stack, so the lemma holds.

For return, all local variables are eliminated so we must show this does not create dangling references. The main reason this is OK is that the call to a procedure only passes values and references. Further (by this lemma, inductively), every reference that is passed is rooted in a variable of type location that is in the calling procedure (or below). Therefore, since (by the static semantics rule) all return values are extensions of references passed as arguments, none of them can be rooted in local variables. Thus no dangling pointers are created when all the local variables are deallocated at a procedure return. $\square$

Proof (Property D). We prove Property D, absence of dangling references (for locations named by variables), by induction on program execution. The base case is an initial call to a procedure, from a state that has no dangling references by construction.

For the induction, suppose program $\mathcal{P}$ is well-typed, state $s = \langle \langle \rho, \ell, L, S \rangle :: P, M \rangle$ satisfies Property L, and $\mathcal{P} \vdash s \rightarrow s'$. The proof proceeds by case analysis on the instruction $\rho[\ell]$.

For a procedure call **Call** $\langle \rho \rangle$, no locations are passed as arguments to Property D holds. (Possible dangling references associated with references passed as arguments are considered in the proof of Property R.)

Ret reduces the domain of memory M' for state s' to $M' = M - \text{img}(L)$. However, by Property U and Property R, no location named by a local variable in L can also be reached from another location variable (Property U) or a reference (Property R is preserved by **Ret**).

StoreLoc $\langle x \rangle$ with $x \notin \text{dom}(L)$ adds local variable x bound to c . However, the domain of memory by $M[c_x \mapsto v]$, preserving Property D.

For **MoveLoc** $\langle x \rangle$, when location c is removed from the domain of memory, this does not produce dangling pointers because of property U – there is no variable referring to c in this case.

No other instructions reduce the domain of memory or add local location variables to the stack. $\square$